

68M 草稿模型与 7B 目标模型推测解码时延实验

实验目的

本实验在 experiments 目录下新增可复现实验脚本，研究 JackFram/llama-68m 草稿模型与 NousResearch/Llama-2-7b-chat-hf 目标模型在同一张 NVIDIA RTX PRO 6000 上进行推测解码时，固定起草长度与文档中熵感知自适应起草长度的时延增益差异。实验暂时忽略通信时延，即令 $T_{comm}=0$ 。

算法口径

实验沿用 docs/System_Model_and_Algorithm.tex 中的有效 token 与 TPOT 定义。若草稿长度为 g ，平均接受率为 α ，则

$$G(\alpha, g) = \sum_{\ell=0}^g \alpha \ell = 1 - \alpha g + 1 - \alpha.$$

实验报告使用观测 TPOT：

$$J_{\sim} = \text{Tround}(g)G_{\sim}, G_{\sim} = K + 1,$$

其中 K 是一轮中被 7B target 接受的 draft token 数。相对 target-only 的时延增益定义为

$$\text{speedup} = J_0 / J_{\sim}, \text{latency reduction} = 1 - J_{\sim} / J_0.$$

固定长度策略分别测试 $g \in \{2, 4, 8\}$ 。自适应策略使用文档中的熵曲线

$$a^{\wedge}(H) = \text{clip}[0, 1](\theta_0 - \theta_1 H),$$

并按预测 TPOT

$$J^{\wedge}_{\ell} = \text{Tround}(\ell)1 + \sum_{k=1}^{\ell} \prod_{r=1}^k a^{\wedge}_r$$

决定是否继续起草。本实验设 $g_{\text{plan}}=8$ 、 $\eta_{\text{acc}}=0.48$ 、 $\theta_0=1.0$ 、 $\theta_1=0.055$ ，近 10 个已验证 token 用于在线拟合熵-接受率曲线。

实验设置

项目	设置
Git 分支	exp/68m-7b-speculative-latency
GPU	NVIDIA RTX PRO 6000 Blackwell Server Edition

项目	设置
Target	experiments/Model/Llama-7B-Chat-Target
Draft	experiments/Model/Llama-68M-Draft
dtype / attention	fp16 / sdpa
数据集	gsm8k, mbpp, wikitext-103-raw-v1
样本数	每个数据集 3 条
Decode 长度	每条样本约 16 个输出 token
通信时延	忽略
结果目录	experiments/speculative_latency_results/run_20260705_162116/

运行命令：

```
OMP_NUM_THREADS=1 /root/miniconda3/envs/SD_Blackwell/bin/python \  
experiments/speculative_latency_experiment.py \  
--datasets gsm8k mbpp wikitext \  
--samples-per-dataset 3 \  
--max-new-tokens 16 \  
--fixed-draft-lengths 2 4 8 \  
--adaptive-plan-g 8 \  
--profile-repeat 2 \  
--output-dir experiments/speculative_latency_results
```

数据集从本机 HuggingFace cache 读取；运行时外网不可达，但三套数据均命中本地缓存。

结果

表中 TPOT 为 wall-clock 平均毫秒/token。speedup 大于 1 表示快于 target-only；latency reduction 为负表示变慢。

数据集	策略	TPOT ms	speedup	latency reduction	接受率	平均上传 g	平均生成 g
gsm8k	target-only	13.65	-	-	-	-	-
gsm8k	fixed g=2	20.36	0.69	-49.2%	47.6%	2.00	2.00
gsm8k	fixed g=4	17.92	0.81	-31.3%	49.8%	4.00	4.00
gsm8k	fixed g=8	19.98	0.74	-46.4%	49.0%	8.00	8.00
gsm8k	adaptive	21.50	0.64	-57.6%	46.3%	1.51	2.51

数据集	策略	TPOT ms	speedup	latency reduction	接受率	平均上传 g	平均生成 g
mbpp	target-only	13.48	-	-	-	-	-
mbpp	fixed g=2	28.47	0.47	-111.2%	10.2%	2.00	2.00
mbpp	fixed g=4	29.04	0.47	-115.4%	13.3%	4.00	4.00
mbpp	fixed g=8	33.13	0.41	-145.7%	12.9%	8.00	8.00
mbpp	adaptive	28.51	0.47	-111.5%	9.1%	1.29	2.29
wikitext	target-only	13.87	-	-	-	-	-
wikitext	fixed g=2	18.25	0.79	-32.1%	55.2%	2.00	2.00
wikitext	fixed g=4	17.54	0.85	-27.1%	51.4%	4.00	4.00
wikitext	fixed g=8	18.38	0.84	-33.4%	54.1%	8.00	8.00
wikitext	adaptive	19.00	0.75	-37.3%	53.8%	1.71	2.67

跨数据集平均：

策略	TPOT ms	speedup	latency reduction	接受率
target-only	13.67	-	-	-
fixed g=2	22.36	0.65	-64.2%	37.6%
fixed g=4	21.50	0.71	-57.9%	38.1%
fixed g=8	23.83	0.66	-75.2%	38.7%
adaptive	23.00	0.62	-68.8%	36.4%

结论

在本次同卡、无通信时延设置下，68M draft + 7B target 没有获得整体正加速。固定长度中 g=4 是三个数据集平均最好的固定策略，但平均 speedup=0.71，仍慢于 target-only。自适应策略能显著降低实际上传长度，例如 gsm8k 平均上传 g=1.51、mbpp 平均上传 g=1.29，但由于草稿 token 必须先生成并计算熵，平均生成长度高于上传长度，额外 draft/cache 维护开销抵消了上传长度下降。

不同数据集差异明显。mbpp 的接受率约 9%-13%，推测解码明显不适合；gsm8k 和 wikitext 的接受率接近或超过 50%，固定 g=4 或 g=8 在个别样本上出现正收益，但按数据集均值仍未超过 target-only。

该结果说明，在通信时延被忽略且两个模型同卡部署时，推测解码能否加速主要取决于接受率是否足够高，以及 draft/cache 管理开销是否足够低。对于论文后续实验，建议进一步比较：更强 draft 模型、batch 化 target 验证、更长输出长度、以及分布式场景下加入通信与终端/边侧异构负载后的 TPOT。

产物

- 实验脚本: `experiments/speculative_latency_experiment.py`
- 原始结果: `experiments/speculative_latency_results/run_20260705_162116/raw_results.csv`
- 汇总结果: `experiments/speculative_latency_results/run_20260705_162116/summary_by_dataset_strategy.csv`
- 元数据: `experiments/speculative_latency_results/run_20260705_162116/metadata.json`